

Ecole

백준 문제 풀이

2025.11.21

Python

구간 합 구하기3



빈출

문제[071] 구간 합 구하기 3

시간 제한 2초 | 난이도 골드 I | 백준 온라인 저지 2042번

어떤 N 개의 수가 주어져 있다. 그런데 중간에 수의 변경이 빈번히 일어나고, 그 중간에 어떤 부분의 합을 구하려 한다. 만약 1, 2, 3, 4, 5라는 수가 있고, 3번째 수를 6으로 바꾸고 2번째부터 5번째까지 합을 구하라고 한다면 17을 출력하면 되는 것이다. 그리고 이 상태에서 5번째 수를 2로 바꾸고, 3번째부터 5번째까지 합을 구하라고 한다면 12가 될 것이다.

예제 입력 1

```
5 2 2 // 노드 개수, 변경 횟수, 구간 합 횟수
1
2
3
4
5
1 3 6
2 2 5
1 5 2
2 3 5
```

예제 출력 1

```
17
12
```

↓ 입력

1번째 줄에 $N(1 \leq N \leq 1,000,000)$ 과 $M(1 \leq M \leq 10,000)$, $K(1 \leq K \leq 10,000)$ 가 주어진다. N 은 수의 개수, M 은 수의 변경이 일어나는 횟수, K 는 구간의 합을 구하는 횟수다. 그리고 2번째 줄부터 $N + 1$ 번째 줄까지 N 개의 수가 주어진다. 그리고 $N + 2$ 번째 줄부터 $N + M + K + 1$ 번째 줄까지 3개의 정수 a , b , c 가 주어지는데, a 가 1일 때 $b(1 \leq b \leq N)$ 번째 수를 c 로 바꾸고, a 가 2일 경우에는 $b(1 \leq b \leq N)$ 번째 수에서 $c(b \leq c \leq N)$ 번째 수까지의 합을 구해 출력하면 된다. 입력으로 주어지는 모든 수는 -263보다 크거나 같고, 263 - 1보다 작거나 같은 정수다.

↑ 출력

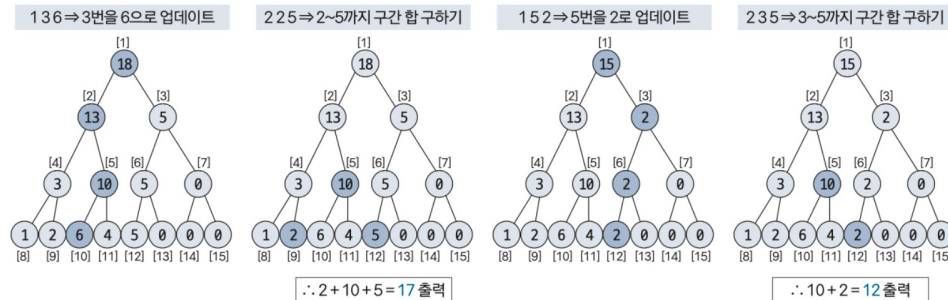
1번째 줄부터 K 줄에 걸쳐 구한 구간의 합을 출력한다. 단, 정답은 -263보다 크거나 같고, 263 - 1보다 작거나 같은 정수다.

**1** 1차원 배열을 이용해 트리의 값을 초기화한다.

트리 배열 크기가 $N = 5$ 이므로 $2^k \geq N$ 을 만족하는 k 의 값은 3이고, 배열의 크기는 $2^3 * 2 = 16$ 이 된다.

시작 인덱스는 $2^3 = \text{start_index} = 8$ 이 된다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	15	10	5	3	7	5	0	1	2	3	4	5	0	0	0

2 질의값 연산 함수와 데이터 업데이트 함수를 수행하고, 결과값을 출력한다.

```

2 import sys
3 input=sys.stdin.readline
4
5 # start, end: 숫자 배열의 인덱스
6 # idx: 세그먼트 트리의 인덱스 (1부터)
7 def make_tree(start, end, idx):
8     if start == end:
9         tree[idx] = arr[start]
10        return tree[idx]
11
12    mid = (start + end) // 2
13    tree[idx] = make_tree(start, mid, idx * 2) + make_tree(mid + 1, end, idx * 2 + 1)
14
15    return tree[idx]
16
17 # target: 수정할 값의 숫자 배열 인덱스
18 # value: 수정할 값 (기존 값에 얼마나 더해야 하는지의 값)
19 def update_tree(start, end, idx, target, value):
20     if target < start or target > end:
21         return
22
23     tree[idx] += value
24     if start == end:
25         return
26
27     mid = (start + end) // 2
28     update_tree(start, mid, idx * 2, target, value)
29     update_tree(mid + 1, end, idx * 2 + 1, target, value)
30
31 # left, right: 구하고자 하는 범위
32 def sum_tree(start, end, idx, left, right):
33     if right < start or left > end:
34         return 0
35
36     if left <= start and right >= end:
37         return tree[idx]
38
39     mid = (start + end) // 2
40     return sum_tree(start, mid, idx * 2, left, right) + sum_tree(mid + 1, end, idx * 2 + 1, left, right)
41
42
43 N, M, K = map(int, input().split())
44 arr = []
45 tree = [0] * (N * 4) # 충분한 트리 공간 확보
46 for _ in range(N):
47     arr.append(int(input()))
48
49 make_tree(0, N-1, 1)
50 for _ in range(M + K):
51     a, b, c = map(int, input().split())
52     if a == 1:
53         update_tree(0, N-1, 1, b-1, c - arr[b-1])
54     else:
55         print(sum_tree(0, N-1, 1, b-1, c-1))
56

```


Java, JS

구간 합 구하기3



Java

구간 합 구하기3 - 정답

Java

```

8 public class P001 {
9     static long[] tree; // 7 usages
10    static long[] arr; // 5 usages
11    static int N; // 7 usages
12
13    // start, end: 숫자 배열의 인덱스 범위
14    // idx: 세그먼트 트리의 현재 노드 인덱스 (1부터 시작)
15    public static long makeTree(int start, int end, int idx) { // 3 usages new *
16        if (start == end) {
17            tree[idx] = arr[start];
18            return tree[idx];
19        }
20
21        int mid = (start + end) / 2;
22        tree[idx] = makeTree(start, mid, idx * 2) + makeTree(start: mid + 1, end, idx: idx * 2 + 1);
23
24        return tree[idx];
25    }
26
27    // target: 수정할 값의 숫자 배열 인덱스 (0-based)
28    // diff: 기존 값에 '얼만큼 더해야 하는지의 값' (변경 후 값 - 변경 전 값)
29    public static void updateTree(int start, int end, int idx, int target, long diff) { // 3 usages new *
30        // target이 현재 노드가 담당하는 범위를 벗어난 경우
31        if (target < start || target > end) {
32            return;
33        }
34
35        // 현재 노드에 diff 반영
36        tree[idx] += diff;
37
38        // 리프 노드에 도달했으면 종료
39        if (start == end) {
40            return;
41        }
42
43        int mid = (start + end) / 2;
44        // 자식 노드 재귀 호출
45        updateTree(start, mid, idx: idx * 2, target, diff);
46        updateTree(start: mid + 1, end, idx: idx * 2 + 1, target, diff);
47    }

```

```

48
49    // left, right: 구하고자 하는 범위의 숫자 배열 인덱스 (0-based)
50    public static long sumTree(int start, int end, int idx, int left, int right) { // 3 usages new *
51        // [start, end]가 [left, right]와 완전히 벗어난 경우
52        if (right < start || left > end) {
53            return 0;
54        }
55
56        // [start, end]가 [left, right]에 완전히 포함되는 경우
57        if (left <= start && right >= end) {
58            return tree[idx];
59        }
60
61        int mid = (start + end) / 2;
62        return sumTree(start, mid, idx: idx * 2, left, right) + sumTree(start: mid + 1, end, idx: idx * 2 + 1, left, right);
63    }
64
65    public static void main(String[] args) throws IOException { // new *
66        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
67        StringBuilder sb = new StringBuilder();
68        StringTokenizer st = new StringTokenizer(br.readLine());
69
70        N = Integer.parseInt(st.nextToken()); // 숫자의 개수
71        int M = Integer.parseInt(st.nextToken()); // 변경 횟수
72        int K = Integer.parseInt(st.nextToken()); // 구간 합 횟수
73
74        arr = new long[N];
75        tree = new long[N * 4];
76
77        // 원본 배열 값 입력
78        for (int i = 0; i < N; i++) {
79            arr[i] = Long.parseLong(br.readLine());
80        }
81
82        makeTree(start: 0, end: N - 1, idx: 1);
83        for (int i = 0; i < M + K; i++) {
84            st = new StringTokenizer(br.readLine());
85            int a = Integer.parseInt(st.nextToken());
86            int b = Integer.parseInt(st.nextToken());
87            long c = Long.parseLong(st.nextToken()); // c는 값 또는 범위 끝 인덱스일 수 있음
88
89            if (a == 1) { // 1: 값 변경
90                int targetIndex = b - 1;
91                long newValue = c;
92
93                // 기존 값과 새로운 값의 차이 (diff)를 계산하여 updateTree 호출
94                long diff = newValue - arr[targetIndex];
95
96                updateTree(start: 0, end: N - 1, idx: 1, targetIndex, diff);
97
98                // 원본 배열 값 변경 (필수)
99                arr[targetIndex] = newValue;
100
101            } else { // 2: 구간 합
102                int left = b - 1;
103                int right = (int) c - 1; // c가 인덱스이므로 int로 변환
104
105                long result = sumTree(start: 0, end: N - 1, idx: 1, left, right);
106                sb.append(result).append("\n");
107            }
108        }
109        System.out.print(sb);
110    }
111 }

```



JS

```
1 const fs = require('fs');
2 const input = fs.readFileSync(0, 'utf8').trim().split('\n');
3 let inputIdx = 0;
4
5 const readLine = () => input[inputIdx++].trim();
6 const [N, M, K] = readLine().split(' ').map(Number);
7
8 const arr = [];
9 for (let i = 0; i < N; i++) {
10   arr.push(BigInt(readLine()));
11 }
12
13 // 세그먼트 트리 배열 (4N 크기, BigInt로 초기화)
14 // 자바스크립트는 배열 크기가 가변적이지만, 메모리 관점에서 미리 4N 크기로 초기화
15 const tree = new Array(N * 4).fill(0n); // 0n은 BigInt 0
16
17 // start, end: 숫자 배열의 인덱스 (0-based)
18 // idx: 세그먼트 트리의 인덱스 (1부터)
19 function makeTree(start, end, idx) {
20   if (start === end) {
21     tree[idx] = arr[start];
22     return tree[idx];
23   }
24
25   const mid = Math.floor((start + end) / 2);
26   // BigInt 덧셈을 사용
27   tree[idx] = makeTree(start, mid, idx * 2n) + makeTree(mid + 1, end, idx * 2n + 1n);
28
29   return tree[idx];
30 }
31
32 // target: 수정할 값의 숫자 배열 인덱스 (0-based)
33 // diff: 기존 값에 '일만큼 더해야 하는지의 값' (BigInt)
34 function updateTree(start, end, idx, target, diff) {
35   if (target < start || target > end) {
36     return;
37   }
38   tree[idx] += diff;
39
40   if (start === end) {
41     return;
42   }
43 }
```

```
44 const mid = Math.floor((start + end) / 2);
45 updateTree(start, mid, idx * 2n, target, diff);
46 updateTree(mid + 1, end, idx * 2n + 1n, target, diff);
47 }
48
49 // left, right: 구하고자 하는 범위의 숫자 배열 인덱스 (0-based)
50 function sumTree(start, end, idx, left, right) {
51   if (right < start || left > end) {
52     return 0n;
53   }
54
55   if (left <= start && right >= end) {
56     return tree[idx];
57   }
58
59   const mid = Math.floor((start + end) / 2);
60   return sumTree(start, mid, idx * 2n, left, right) + sumTree(mid + 1, end, idx * 2n + 1n, left, right);
61 }
62
63 // 초기 세그먼트 트리 생성
64 makeTree(0, N - 1, 1n); // 루트 노드 인덱스 1n (BigInt)
65
66 const result = [];
67 // M + K 번의 연산 처리
68 for (let i = 0; i < M + K; i++) {
69   const line = readLine().split(' ');
70   // a, b는 숫자, c는 BigInt로 처리
71   const a = Number(line[0]);
72   const b = Number(line[1]);
73   const c = BigInt(line[2]);
74
75   if (a === 1) { // 1: 값 변경
76     const targetIndex = b - 1; // 0-based 인덱스
77     const newValue = c;
78
79     // 기존 값과 새로운 값의 차이 (diff)를 계산하여 updateTree 호출
80     const diff = newValue - arr[targetIndex];
81
82     // idx를 BigInt로 전달 (1n)
83     updateTree(0, N - 1, 1n, targetIndex, diff);
84
85     // 원본 배열 값 변경 (필수)
86     arr[targetIndex] = newValue;
87   } else { // 2: 구간 합
88     const left = b - 1;
89     const right = Number(c) - 1; // c가 인덱스이므로 Number로 변환
90
91     const sum = sumTree(0, N - 1, 1n, left, right);
92     result.push(sum.toString()); // 결과를 문자열로 저장
93   }
94 }
95 console.log(result.join('\n'));
```

Python

최소 공통 조상 구하기1

**문제[074] 최소 공통 조상 구하기 1**

시간 제한 3초 | 난이도 골드 III | 백준 온라인 저지 11437번

$N(2 \leq N \leq 50,000)$ 개의 노드로 이루어진 트리가 주어진다. 트리의 각 노드는 1번부터 N 번까지 번호가 매겨져 있으며, 루트는 1번이다. 두 노드의 쌍 $M(1 \leq M \leq 10,000)$ 개가 주어졌을 때 두 노드의 가장 가까운 공통 조상이 몇 번인지 출력하시오.

↓ 입력

1번째 줄에 노드의 개수 N , 그다음 $N - 1$ 개 줄에는 트리상에서 연결된 두 노드가 주어진다. 그다음 줄에 가장 가까운 공통 조상을 알고 싶은 쌍의 개수 M 이 주어지고, 그다음 M 개 줄에는 노드 쌍이 주어진다.

↑ 출력

M 개의 줄에 차례대로 입력받은 두 노드의 가장 가까운 공통 조상을 출력한다.

예제 입력 1

```
15 // 노드 개수
1 2
1 3
2 4
3 7
6 2
3 8
4 9
2 5
5 11
7 13
10 4
11 15
12 5
14 7
6 // 질의 개수
6 11
10 9
2 6
7 6
8 13
8 15
```

예제 출력 1

```
2
4
2
1
3
1
```



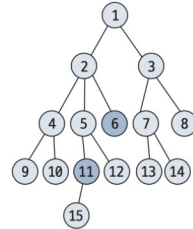
Python

최소 공통 조상 구하기1 - 정답

1 인접 리스트로 트리 데이터를 구현한다.

2 탐색 알고리즘(DFS,BFS)을 이용해 각 노드의 깊이를 구한다.

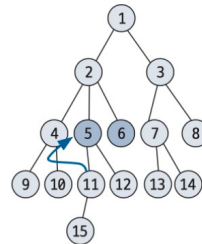
index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
부모 노드	0	1	1	2	2	2	3	3	4	4	5	5	7	7	11
깊이	0	1	1	2	2	2	2	2	3	3	3	3	3	3	4



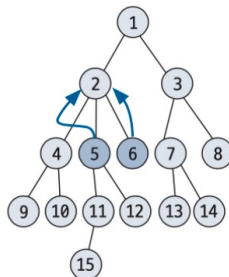
3 깊이를 맞추기 위해 더 깊은 노드를 같은 깊이가 될 때까지 부모 노드로 이동한다.

깊이가 1만큼 차이므로 깊이가 3인 11번 노드를 부모 노드로 한 번 업데이트 한다.

index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
부모 노드	0	1	1	2	2	2	3	3	4	4	5	5	7	7	11
깊이	0	1	1	2	2	2	2	2	3	3	3	3	3	3	4



4 부모 노드로 계속 올라가면서 최소 공통 조상을 찾는다.



LCA(6, 11) = 2 ⇒ ∴ 2 출력

Python

```
import sys
sys.setrecursionlimit(100000)

n = int(input())
parent = [0] * (n + 1)  # 각 노드의 부모 노드 정보
d = [0] * (n + 1)  # 각 노드까지의 깊이
visited = [0] * (n + 1)  # 방문 여부
graph = [[] for _ in range(n + 1)]

for _ in range(n - 1):
    a, b = map(int, input().split())
    graph[a].append(b)
    graph[b].append(a)

# 루트 노드부터의 깊이 구하기
def dfs(x, depth):
    visited[x] = True
    d[x] = depth

    for node in graph[x]:
        if visited[node]:
            continue
        parent[node] = x
        dfs(node, depth + 1)

# 최소 공통 조상 찾기
def lca(a, b):
    # 깊이 맞추기
    while d[a] != d[b]:
        if d[a] > d[b]:
            a = parent[a]
        else:
            b = parent[b]

    # 노드 맞추기
    while a != b:
        a = parent[a]
        b = parent[b]

    return a

dfs[1, 0]
m = int(input())
for _ in range(m):
    a, b = map(int, input().split())
    print(lca(a, b))
```

Java, JS

최소 공통 조상 구하기1



Java

```

public class P002 {
    static int N;
    static int[] parent;
    static int[] depth;
    static ArrayList<ArrayList<Integer>> graph;

    public static void dfs(int curr, int dep, int pcurr) {
        depth[curr] = dep;
        parent[curr] = pcurr;

        for (int next : graph.get(curr)) {
            // 부모 노드가 아닌 경우에만 재귀 호출 (무방향 그래프 탐색 시 중요)
            if (next != pcurr) {
                dfs(next, dep + 1, curr);
            }
        }
    }

    public static int lca(int a, int b) {
        // 1. 길이 맞추기 (항상 b가 더 깊거나 같은 깊이가 되도록 swap)
        if (depth[a] > depth[b]) {
            int temp = a;
            a = b;
            b = temp;
        }

        // 깊이가 같아질 때까지 깊이가 더 깊은 노드(b)를 부모 노드로 이동
        while (depth[a] != depth[b]) {
            b = parent[b];
        }

        // 2. 노드 맞추기 (두 노드가 같아질 때까지 동시에 부모 노드로 이동)
        while (a != b) {
            a = parent[a];
            b = parent[b];
        }

        return a;
    }
}

```

```

public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st;
    StringBuilder sb = new StringBuilder();

    N = Integer.parseInt(br.readLine());

    // 배열 및 리스트 초기화 (노드 번호 1부터 N까지 사용)
    parent = new int[N + 1];
    depth = new int[N + 1];
    graph = new ArrayList<>();
    for (int i = 0; i <= N; i++) {
        graph.add(new ArrayList<>());
    }

    // 트리의 연결 정보 입력 (N-1개의 간선)
    for (int i = 0; i < N - 1; i++) {
        st = new StringTokenizer(br.readLine());
        int u = Integer.parseInt(st.nextToken());
        int v = Integer.parseInt(st.nextToken());
        graph.get(u).add(v);
        graph.get(v).add(u); // 양방향 연결
    }

    // DFS를 통해 깊이와 부모 노드 설정
    // 루트 노드를 1로 가정, 깊이 0부터 시작, 부모 노드는 0
    dfs(1, 0, 0);

    // LCA 질의 개수 입력
    int M = Integer.parseInt(br.readLine());

    // M번의 LCA 질의 처리
    for (int i = 0; i < M; i++) {
        st = new StringTokenizer(br.readLine());
        int u = Integer.parseInt(st.nextToken());
        int v = Integer.parseInt(st.nextToken());
        sb.append(lca(u, v)).append("\n");
    }

    System.out.print(sb);
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync(0, 'utf8').trim().split('\n');
let inputIdx = 0;
const readLine = () => input[inputIdx++].trim().split(' ').map(Number);
const N = readLine()[0];
const parent = new Array(N + 1).fill(0); // 각 노드의 부모 노드
const depth = new Array(N + 1).fill(0); // 각 노드까지의 깊이
const graph = new Array(N + 1).fill(0).map(() => []); // 인접 리스트

for (let i = 0; i < N - 1; i++) {
    const [a, b] = readLine();
    graph[a].push(b);
    graph[b].push(a); // 양방향 연결
}

function dfs(curr, dep, pcurr) {
    depth[curr] = dep;
    parent[curr] = pcurr;

    for (const next of graph[curr]) {
        // 부모 노드가 아닌 경우에만 탐색
        if (next !== pcurr) {
            dfs(next, dep + 1, curr);
        }
    }
}

function lca(a, b) {
    if (depth[a] > depth[b]) {
        [a, b] = [b, a]; // swap
    }

    while (depth[a] !== depth[b]) {
        b = parent[b];
    }

    while (a !== b) {
        a = parent[a];
        b = parent[b];
    }

    return a;
}

dfs(1, 0, 0);
const M = readLine()[0];
const result = [];
for (let i = 0; i < M; i++) {
    const [a, b] = readLine();
    result.push(lca(a, b));
}

console.log(result.join('\n'));

```


Python

최소 공통 조상 구하기2



빈출

문제[075] 최소 공통 조상 구하기 2

시간 제한 1.5초 | 난이도 플래티넘 | 백준 온라인 저지 11438번

N ($2 \leq N \leq 100,000$)개의 노드로 이뤄진 트리가 주어진다. 트리의 각 노드는 1번부터 N 번까지 번호가 매겨져 있으며, 루트는 1번이다. 두 노드의 쌍 M ($1 \leq M \leq 100,000$)개가 주어졌을 때 두 노드의 가장 가까운 공통 조상이 몇 번인지 출력하시오.

↓ 입력

1번째 줄에 노드의 개수 N , 그다음 $N - 1$ 개 줄에는 트리상에서 연결된 두 노드가 주어진다. 그다음 줄에 가장 가까운 공통 조상을 알고 싶은 쌍의 개수 M 이 주어지고, 그다음 M 개의 줄에는 노드 쌍이 주어진다.

↑ 출력

M 개의 줄에 차례대로 입력받은 두 노드의 가장 가까운 공통 조상을 출력한다.

예제 입력 1

```
15 // 노드 개수
1 2
1 3
2 4
3 7
6 2
3 8
4 9
2 5
5 11
7 13
```

```
10 4
11 15
12 5
14 7
6 // 질의 개수
6 11
10 9
2 6
7 6
8 13
8 15
```

예제 출력 1

```
2
4
2
1
3
1
```



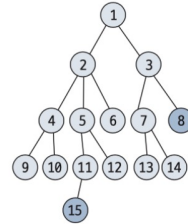
Python

최소 공통 조상 구하기2 - 정답

- 1 인접 리스트로 트리 데이터를 구현한다.
- 2 탐색 알고리즘(DFS, BFS)을 이용해 각 노드의 깊이를 구한다.

Depth(깊이 배열)

index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
깊이	0	1	1	2	2	2	2	2	3	3	3	3	3	3	4



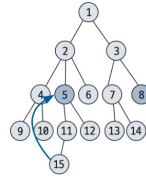
- 3 점화식을 이용해 Parent 배열(부모 노드 배열)을 구한다.

부모 노드 배열 점화식

$$P[K][N] = P[K - 1][P[K - 1][N]]$$

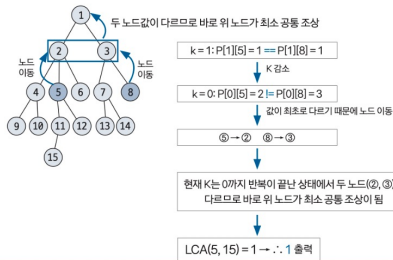
Parent(부모 배열)

index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2 ⁰ 번째 부모 노드	0	1	1	2	2	2	3	3	4	4	5	5	7	7	11
2 ¹ 번째 부모 노드	0	0	0	1	1	1	1	1	2	2	2	2	3	3	5
2 ² 번째 부모 노드	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1



- 4 깊이가 큰 노드는 Parent 배열을 이용해 2^k 만큼 빠르게 이동시켜 깊이를 맞춘다.
깊이가 2만큼 차이(2^1)가 나므로 15번 노드를 2¹번째 부모 노드인 5로 변경해 깊이를 맞춘다.
한 칸씩 오르는 것이 아닌 2의 제곱수로 빠르게 올라가게 된다.

- 5 부모 노드로 올라가면서 최소 공통 조상을 찾는다.



Python

```
# 11438 최소 공통 조상 구하기2
import sys
# 재귀 한도 설정 (이미 되어 있음)
# sys.setrecursionlimit(100000)

# 입력을 한 번에 읽어와서 줄 단위로 분리하고 처리하는 것이 가장 빠릅니다.
def solve():
    # 모든 입력을 읽어와서 줄 단위로 분리
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # 입력 포인터 초기화
    data_idx = 0
    def read_int():
        nonlocal data_idx
        val = int(input_data[data_idx])
        data_idx += 1
        return val

    # 트리 크기 N 입력
    n = read_int()

    # 2^i <= N을 만족하는 최대 i + 1. N=100000일 때 log2(100000) ~ 16.6, 21은 충분함.
    # log2(N) + 1 로 계산하는 것이 더 정확합니다.
    LENGTH = 0
    if n > 0:
        LENGTH = n.bit_length() # log2(N) + 1

    # 배열 초기화
    parent = [[0] * LENGTH for _ in range(n + 1)]
    d = [0] * (n + 1)
    graph = [[] for _ in range(n + 1)]

    # 간선 정보 입력
    for _ in range(n - 1):
        a = read_int()
        b = read_int()
        graph[a].append(b)
        graph[b].append(a)

    # DFS를 통해 깊이와 2^0번째 부모를 설정
    stack = [(1, 0, 0)] # (노드, 깊이, 부모)
    visited = [False] * (n + 1)
```

```
# 재귀 대신 반복문(Stack) 기반 DFS 사용 (파이썬 재귀 깊이 제한 및 오버헤드 회피)
while stack:
    x, depth, p = stack.pop()

    if visited[x]:
        continue
    visited[x] = True
    d[x] = depth
    parent[x][0] = p # 현재 부모 저장

    for node in graph[x]:
        if not visited[node]:
            # 다음 탐색을 위해 스택에 추가 (깊이 + 1, 현재 노드를 부모로)
            stack.append((node, depth + 1, x))

# 최소 배열 부모 관계 갱신
for i in range(1, LENGTH):
    for j in range(1, n + 1):
        parent[j][i] = parent[parent[j][i - 1]][i - 1]

# LCA 함수
def lca(a, b):
    # 1. b의 깊이가 더 깊도록 설정
    if d[a] > d[b]:
        a, b = b, a

    # 2. 깊이 맞추기 (O(log N))
    diff = d[b] - d[a]
    for i in range(LENGTH):
        if (diff >> i) & 1:
            b = parent[b][i]

    if a == b:
        return a

    # 3. 올라가면서 공통 조상 찾기 (O(log N))
    for i in range(LENGTH - 1, -1, -1):
        if parent[a][i] != parent[b][i]:
            a = parent[a][i]
            b = parent[b][i]

    return parent[a][0]

# LCA 점의 개수 M
m = read_int()

# 출력을 모으기 위한 리스트
results = []

# M번의 LCA 점의 처리
for _ in range(m):
    a = read_int()
    b = read_int()
    results.append(str(lca(a, b)))

# 한 번에 출력
sys.stdout.write('\n'.join(results) + '\n')

solve()
```

Java, JS

최소 공통 조상 구하기2



Java

최소 공통 조상 구하기2 - 정답

Java

```
public class P003 { & kim-do-kyun *
    static int N; // 노드의 개수 9 usages
    static int LENGTH; // 최소 배열의 크기 (log2(N) + 1) 6 usages
    static int[][] parent; // parent[노드][i]: 노드의 2^i번째 부모 11 usages
    static int[] depth; // depth[노드]: 루트로부터의 깊이 6 usages
    static ArrayList<ArrayList<Integer>> graph; // 인접 리스트로 그래프 표현 5 usages

    public static void calculateBaseInfo() { 1 usage & kim-do-kyun
        // (현재 노드, 깊이, 부모 노드)
        Stack<int[]> stack = new Stack<>();
        stack.push(new int[]{1, 0, 0});

        boolean[] visited = new boolean[N + 1];

        while (!stack.isEmpty()) {
            int[] current = stack.pop();
            int curr = current[0];
            int dep = current[1];
            int p = current[2];

            if (visited[curr]) continue;
            visited[curr] = true;

            depth[curr] = dep;
            parent[curr][0] = p; // 1번째 부모 저장

            for (int next : graph.get(curr)) {
                // 부모 노드가 아닌 경우에만 스택에 추가
                if (next != p) {
                    stack.push(new int[]{next, dep + 1, curr});
                }
            }
        }
    }
}
```

```
public static void fillSparseArray() { 1 usage & kim-do-kyun
    for (int i = 1; i < LENGTH; i++) {
        for (int j = 1; j <= N; j++) {
            // parent[j][i] = parent[ parent[j][i-1] ][i-1]
            int intermediateParent = parent[j][i - 1];
            parent[j][i] = parent[intermediateParent][i - 1];
        }
    }
}

public static int lca(int a, int b) { 1 usage & kim-do-kyun
    // 1. b의 깊이가 더 깊거나 같도록 설정
    if (depth[a] > depth[b]) {
        int temp = a;
        a = b;
        b = temp;
    }

    // 2. 깊이 맞추기 (O(log N))
    int diff = depth[b] - depth[a];
    for (int i = 0; i < LENGTH; i++) {
        // diff의 i번째 비트가 1이면 2^i만큼 점프
        if ((diff & (1 << i)) > 0) {
            b = parent[b][i];
        }
    }

    // 깊이가 같아진 후 두 노드가 같다면 바로 공통 조상
    if (a == b) {
        return a;
    }

    // 3. 공통 조상의 바로 아래까지 동시에 올라가기 (O(log N))
    for (int i = LENGTH - 1; i >= 0; i--) {
        // 두 노드의 2^i 번째 부모가 다르다면
        if (parent[a][i] != parent[b][i]) {
            // 동시에 점프
            a = parent[a][i];
            b = parent[b][i];
        }
    }
}
```

```
// LCA 바로 아래에 위치한 두 노드의 부모가 LCA
return parent[a][0];
}

public static void main(String[] args) throws IOException { & kim-do-kyun
    // 빠른 입출력을 위해 BufferedReader 사용
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringBuilder sb = new StringBuilder();
    StringTokenizer st;

    // N 입력
    N = Integer.parseInt(br.readLine());

    // LENGTH 계산 (N이 1 이상일 경우)
    if (N > 0) {
        LENGTH = (int) Math.ceil(Math.log(N) / Math.log(2)) + 1;
    } else {
        LENGTH = 1;
    }

    // 배열 및 리스트 초기화 (1-based index)
    parent = new int[N + 1][LENGTH];
    depth = new int[N + 1];
    graph = new ArrayList<>();
    for (int i = 0; i <= N; i++) {
        graph.add(new ArrayList<>());
    }

    // 간선 정보 입력
    for (int i = 0; i < N - 1; i++) {
        st = new StringTokenizer(br.readLine());
        int u = Integer.parseInt(st.nextToken());
        int v = Integer.parseInt(st.nextToken());
        graph.get(u).add(v);
        graph.get(v).add(u); // 양방향 연결
    }

    // 전처리의 깊이 및 최소 배열 계산
    calculateBaseInfo();
    fillSparseArray();

    // LCA 질의 개수 M 입력
    int M = Integer.parseInt(br.readLine());

    // M번의 LCA 질의 처리
    for (int i = 0; i < M; i++) {
        st = new StringTokenizer(br.readLine());
        int u = Integer.parseInt(st.nextToken());
        int v = Integer.parseInt(st.nextToken());
        sb.append(lca(u, v)).append("\n");
    }

    // 결과 한 번에 출력
    System.out.print(sb);
}
}
```



JS

```
const fs = require('fs');

function getOptimizedInput() {
    // 모든 입력을 읽어와서 공백 기준으로 분리
    const inputData = fs.readFileSync(0, 'utf8').trim().split(/\s+/).map(Number);
    let dataIdx = 0;

    // 다음 정수 토큰을 읽는 함수. 이 함수를 통해 N, 간선, M, 쿼리를 순차적으로 읽습니다.
    const readInt = () => inputData[dataIdx++];

    return { readInt };
}

function solve() {
    const { readInt } = getOptimizedInput();

    // [수정] N을 readInt()로 소비해야 다음 입력부터 간선 정보가 시작됩니다.
    const N = readInt();

    if (N <= 0) return;

    // LENGTH: 2^k 번째 부모를 저장하기 위한 최대 길이 (log2(N) + 1)
    // N.toString(2).length는 N의 이진수 길이를 반환하며, 이는 ceil(log2(N+1))과 같습니다.
    const LENGTH = N.toString(2).length;

    // 배열 초기화 (1-based index)
    const parent = Array(N + 1).fill(0).map(() => Array(LENGTH).fill(0));
    const depth = Array(N + 1).fill(0);
    const graph = Array(N + 1).fill(0).map(() => []);

    // 1. 그래프 구성
    for (let i = 0; i < N - 1; i++) {
        const u = readInt();
        const v = readInt();
        graph[u].push(v);
        graph[v].push(u);
    }

    // 2. DFS를 통한 기본 정보(depth, 1번째 부모) 계산
    function calculateBaseInfo() {
        // Stack 기반 DFS: [노드, 깊이, 부모]
        const stack = [[1, 0, 0]]; // 루트 노드 1, 깊이 0, 부모 0
        const visited = Array(N + 1).fill(false);

        while (stack.length > 0) {
            const [curr, dep, p] = stack.pop();

            if (visited[curr]) continue;
            visited[curr] = true;

            depth[curr] = dep;
            parent[curr][0] = p; // 1번째 부모 저장

            for (const next_node of graph[curr]) {
                if (!visited[next_node]) {
                    stack.push([next_node, dep + 1, curr]);
                }
            }
        }
    }

    // 3. 최소 배열(Sparse Array) 채우기: 2^i 번째 부모 계산
    function fillSparseArray() {
        for (let i = 1; i < LENGTH; i++) {
            for (let j = 1; j <= N; j++) {
                // parent[j][i] = parent[ parent[j][i-1] ][i-1]
                const intermediateParent = parent[j][i - 1];
                parent[j][i] = parent[intermediateParent][i - 1];
            }
        }
    }

    // 4. 최소 공통 조상(LCA) 함수
    function lca(a, b) {
        // 무조건 b의 깊이가 더 깊거나 같도록 설정 (구조 분해 할당으로 스왑)
        if (depth[a] > depth[b]) {
            [a, b] = [b, a];
        }

        // 1. 깊이 맞추기 (O(log N))
        let diff = depth[b] - depth[a];
        for (let i = 0; i < LENGTH; i++) {
            // diff의 i번째 비트가 1이면 2^i만큼 점프
            if ((diff >> i) & 1) {
                b = parent[b][i];
            }
        }

        // 2. 공통 조상의 바로 아래까지 동시에 올라가기 (O(log N))
        for (let i = LENGTH - 1; i >= 0; i--) {
            // 두 노드의 2^i 번째 부모가 다르면 동시에 점프
            if (parent[a][i] !== parent[b][i]) {
                a = parent[a][i];
                b = parent[b][i];
            }
        }

        // LCA 바로 아래에 위치한 두 노드의 부모가 LCA
        return parent[a][0];
    }

    // 전처리 실행
    calculateBaseInfo();
    fillSparseArray();

    // 5. LCA 질의 처리
    const M = readInt(); // 쿼리 개수

    const results = [];
    for (let i = 0; i < M; i++) {
        const u = readInt();
        const v = readInt();
        results.push(lca(u, v));
    }

    // 결과를 한 번에 출력
    console.log(results.join('\n'));
}

solve();
```

```
while (stack.length > 0) {
    const [curr, dep, p] = stack.pop();

    if (visited[curr]) continue;
    visited[curr] = true;

    depth[curr] = dep;
    parent[curr][0] = p; // 1번째 부모 저장

    for (const next_node of graph[curr]) {
        if (!visited[next_node]) {
            stack.push([next_node, dep + 1, curr]);
        }
    }
}

// 3. 최소 배열(Sparse Array) 채우기: 2^i 번째 부모 계산
function fillSparseArray() {
    for (let i = 1; i < LENGTH; i++) {
        for (let j = 1; j <= N; j++) {
            // parent[j][i] = parent[ parent[j][i-1] ][i-1]
            const intermediateParent = parent[j][i - 1];
            parent[j][i] = parent[intermediateParent][i - 1];
        }
    }
}

// 4. 최소 공통 조상(LCA) 함수
function lca(a, b) {
    // 무조건 b의 깊이가 더 깊거나 같도록 설정 (구조 분해 할당으로 스왑)
    if (depth[a] > depth[b]) {
        [a, b] = [b, a];
    }

    // 1. 깊이 맞추기 (O(log N))
    let diff = depth[b] - depth[a];
    for (let i = 0; i < LENGTH; i++) {
        // diff의 i번째 비트가 1이면 2^i만큼 점프
        if ((diff >> i) & 1) {
            b = parent[b][i];
        }
    }

    // 2. 공통 조상의 바로 아래까지 동시에 올라가기 (O(log N))
    for (let i = LENGTH - 1; i >= 0; i--) {
        // 두 노드의 2^i 번째 부모가 다르면 동시에 점프
        if (parent[a][i] !== parent[b][i]) {
            a = parent[a][i];
            b = parent[b][i];
        }
    }

    // LCA 바로 아래에 위치한 두 노드의 부모가 LCA
    return parent[a][0];
}
```

```

    }

    if (a === b) {
        return a;
    }

    // 2. 공통 조상의 바로 아래까지 동시에 올라가기 (O(log N))
    for (let i = LENGTH - 1; i >= 0; i--) {
        // 두 노드의 2^i 번째 부모가 다르면 동시에 점프
        if (parent[a][i] !== parent[b][i]) {
            a = parent[a][i];
            b = parent[b][i];
        }
    }

    // LCA 바로 아래에 위치한 두 노드의 부모가 LCA
    return parent[a][0];
}

// 전처리 실행
calculateBaseInfo();
fillSparseArray();

// 5. LCA 질의 처리
const M = readInt(); // 쿼리 개수

const results = [];
for (let i = 0; i < M; i++) {
    const u = readInt();
    const v = readInt();
    results.push(lca(u, v));
}

// 결과를 한 번에 출력
console.log(results.join('\n'));
}

solve();
```

Python

이항계수 구하기1

**문제[076] 이항계수 구하기 1**

시간 제한 1초 | 난이도 브론즈 1 | 백준 온라인 저지 11050번

자연수 N 과 정수 K 가 주어졌을 때 이항계수 $\binom{n}{k}$ 를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N 과 K 가 주어진다($1 \leq N \leq 10, 0 \leq K \leq N$).

↑ 출력

$\binom{n}{k}$ 를 출력한다.

예제 입력 1

5 2 // N K

예제 출력 1

10



Python

이항계수 구하기1 - 정답

1 N과 K의 값을 입력 받고 DP 배열을 선언한다. ($D[N + 1][N + 1]$)

DP 배열의 값을 다음과 같이 초기화 한다.

DP 배열 초기화

$D[i][1] = i$ // i 개 중 1개를 뽑는 경우의 수는 i 개

$D[i][0] = 1$ // i 개 중 1개도 선택하지 않는 경우의 수는 1개

$D[i][i] = 1$ // i 개 중 i 개를 선택하는 경우의 수는 1개

$k \leftarrow$ 선택 수

1	0	0	0	0	0
1	1	0	0	0	0
1	2	1	0	0	0
1	3	0	1	0	0
1	4	0	0	1	0
1	5	0	0	0	1

$N \leftarrow$ 총 개수

총 개수보다 선택 수가 더 클 수 없으므로 이 부분은 처리하지 않음

2 점화식으로 DP 배열의 값을 채운다.

조합 점화식

$$D[i][j] = D[i - 1][j] + D[i - 1][j - 1]$$

1	0	0	0	0	0
1	1	0	0	0	0
1	2	1	0	0	0
1	3	3	1	0	0
1	4	6	4	1	0
1	5	10	10	5	1

3 $D[N][K]$ 의 값을 출력한다.

1	0	0	0	0	0
1	1	0	0	0	0
1	2	1	0	0	0
1	3	3	1	0	0
1	4	6	4	1	0
1	5	10	10	5	1

$\therefore D[5][2] = 10$

Python

```
import sys

input = sys.stdin.readline

# N, K 입력
N, K = map(int, input().split())

# DP 테이블 초기화
DP = [[0] * (N + 1) for _ in range(N + 1)]

# 파스칼의 삼각형을 이용한 DP 테이블 채우기
for i in range(N + 1):
    DP[i][0] = 1 # nC0는 항상 1
    DP[i][i] = 1 # nCn은 항상 1

for i in range(2, N + 1):
    for j in range(1, i):
        DP[i][j] = (DP[i - 1][j - 1] + DP[i - 1][j]) % 10007 # 중간 값들도 모듈러 연산 적용

# 원하는 형식으로 출력
print(DP[N][K] % 10007)
```

Java, JS

이항계수 구하기1



Java

```

public class P004 {
    static final int MOD = 10007;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        // N, K 입력
        int N = Integer.parseInt(st.nextToken());
        int K = Integer.parseInt(st.nextToken());

        // K가 유효하지 않은 범위일 경우 처리
        if (K < 0 || K > N) {
            System.out.println(0);
            return;
        }

        // DP 테이블 초기화. DP[i][j]는 iCj (i choose j)의 결과
        int[][] DP = new int[N + 1][N + 1];

        // 파스칼의 삼각형을 이용한 DP 테이블 채우기
        for (int i = 0; i <= N; i++) {
            DP[i][0] = 1; // nC0는 항상 1

            // iC1부터 iC(i-1)까지 계산
            for (int j = 1; j < i; j++) {
                // 파스칼의 항등식: iCj = (i-1)C(j-1) + (i-1)Cj
                // 모든 중간 결과에 모듈러 연산 적용
                DP[i][j] = (DP[i - 1][j - 1] + DP[i - 1][j]) % MOD;
            }

            DP[i][i] = 1; // nCn은 항상 1
        }

        // 최종 결과 출력
        System.out.println(DP[N][K]);
    }
}

```

JS

```

const fs = require('fs');

// 모듈러 연산 상수
const MOD = 10007;

function solve() {
    // 입력을 읽고 N과 K를 추출
    const input = fs.readFileSync(0, 'utf8').trim().split(/\s+/);

    const N = parseInt(input[0], 10);
    const K = parseInt(input[1], 10);

    // 유효성 검사 및 예외 처리
    if (K < 0 || K > N || isNaN(N) || isNaN(K)) {
        console.log(0);
        return;
    }

    // DP 테이블 초기화. DP[i][j]는 iCj의 결과를 저장
    // 크기 (N+1) x (N+1)
    const DP = Array(N + 1).fill(0).map(() => Array(N + 1).fill(0));

    // 파스칼의 삼각형을 이용한 DP 테이블 채우기
    for (let i = 0; i <= N; i++) {
        DP[i][0] = 1; // nC0 = 1

        for (let j = 1; j < i; j++) {
            // 파스칼의 항등식: iCj = (i-1)C(j-1) + (i-1)Cj
            // 모든 중간 결과에 모듈러 연산 적용
            DP[i][j] = (DP[i - 1][j - 1] + DP[i - 1][j]) % MOD;
        }

        DP[i][i] = 1; // nCn = 1
    }

    // 최종 결과 출력
    console.log(DP[N][K]);
}

solve();

```

Python

이항계수 구하기2

**문제[077] 이항계수 구하기 2**

시간 제한 1초 | 난이도 실버 I | 백준 온라인 저지 11051번

자연수 N 과 정수 K 가 주어졌을 때 이항계수 $\binom{n}{k}$ 를 10,007로 나눈 나머지를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N 과 K 가 주어진다($1 \leq N \leq 1,000, 0 \leq K \leq N$).

↑ 출력

$\binom{n}{k}$ 를 출력한다.

예제 입력 1

```
5 2 // N K
```

예제 출력 1

```
10
```



Python

이항계수 구하기2 - 정답

- 1 N과 K의 값을 입력받은 후 DP 배열을 선언한다. ($D[N+1][N+1]$).
DP 배열의 값을 다음과 같이 초기화한다.

DP 배열 초기화

$D[i][1] = i$ // i 개 중 1개를 뽑는 경우의 수는 i 개
 $D[i][0] = 1$ // i 개 중 1개도 선택하지 않는 경우의 수는 1개
 $D[i][i] = 1$ // i 개 중 i 개를 선택하는 경우의 수는 1개

k ← 선택 수

1	0	0	0	0	0
1	1	0	0	0	0
1	2	1	0	0	0
1	3	0	1	0	0
1	4	0	0	1	0
1	5	0	0	0	1

N ← 총 개수

- 2 점화식으로 DP 배열의 값을 채운다. 점화식의 결과값이 나올 때마다 MOD 연산을 수행한다.

조합 점화식

$$D[i][j] = D[i-1][j] + D[i-1][j-1]$$

1	0	0	0	0	0
1	1	0	0	0	0
1	2	1	0	0	0
1	3	3	1	0	0
1	4	6	4	1	0
1	5	10	10	5	1

점화식의 결과가 나올 때마다 모듈러 연산 수행

예) $D[5][2] = D[4][2] + D[4][1]$
 $\Rightarrow D[5][2] = D[5][2] \% 10,007$

- 3 $D[N][K]$ 의 값을 출력한다.

$$\therefore D[5][2] = 10$$

Python

```
import sys
input = sys.stdin.readline

N, K = map(int, input().split())

# 파스칼 삼각형을 활용한 DP 풀이 (nCk = (n-1)C(k-1) + (n-1)Ck)
# 비어있는 파스칼 삼각형 구축해놓기
pascal = [0]

for depth in range(2, N+2):
    pascal.append([0]*depth)

# pascal[i] = [1, ?, ?, ..., ?, 1]에서 i는 nCk에서 n, 리스트의 인덱스는 k를 의미
# nC0 = 1
pascal[1] = [1, 1]

for depth in range(2, N):
    pascal[depth][0] = 1
    for idx in range(1, depth):
        pascal[depth][idx] = (pascal[depth-1][idx-1] + pascal[depth-1][idx]) % 10007
    pascal[depth][-1] = 1

if K == 0 or K == N:
    print(1)
else:
    print((pascal[N-1][K-1] + pascal[N-1][K]) % 10007)
```

Java, JS

이항계수 구하기2



Java

```
public class P005 { @ kim-do-kyun *
    public static void main(String[] args) throws IOException { @ kim-do-kyun *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int N = Integer.parseInt(st.nextToken());
        int K = Integer.parseInt(st.nextToken());

        // K가 0이거나 N인 경우 이항 계수는 항상 1
        if (K == 0 || K == N) {
            System.out.println(1);
            return;
        }

        int[][] pascal = new int[N + 1][];

        if (N > 1) {
            pascal[1] = new int[]{1, 1};
        }

        // DP 수행
        for (int depth = 2; depth < N; depth++) {
            // 해당 depth(행)의 크기는 depth + 1이어야 함 (예: 2번째 행은 1, 2, 1로 3개)
            pascal[depth] = new int[depth + 1];

            pascal[depth][0] = 1; // 첫 번째 원소

            // 중간 원소를 계산
            for (int idx = 1; idx < depth; idx++) {
                pascal[depth][idx] = (pascal[depth - 1][idx - 1] + pascal[depth - 1][idx]) % 10007;
            }

            pascal[depth][depth] = 1; // 마지막 원소
        }

        int result = (pascal[N - 1][K - 1] + pascal[N - 1][K]) % 10007;
        System.out.println(result);
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split(' ');

const N = parseInt(input[0]);
const K = parseInt(input[1]);

if (K === 0 || K === N) {
    console.log(1);
} else {
    // 파스칼 삼각형 배열 초기화
    let pascal = [];

    // n=1 일 때 [1, 1]
    pascal[1] = [1, 1];

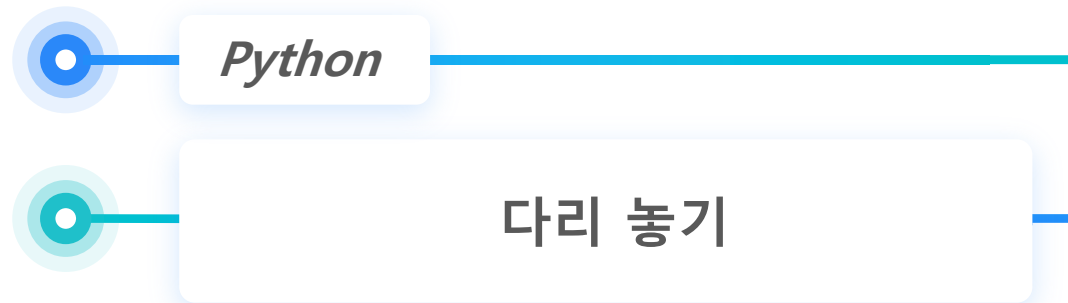
    for (let depth = 2; depth < N; depth++) {
        // 자바스크립트는 배열 크기를 미리 지정하지 않아도 되지만,
        // 명시적으로 depth + 1 크기의 배열을 생성하는 것이 로직상 안전함
        pascal[depth] = new Array(depth + 1);

        pascal[depth][0] = 1; // 첫 원소

        for (let idx = 1; idx < depth; idx++) {
            // 모듈러 연산 적용
            pascal[depth][idx] = (pascal[depth - 1][idx - 1] + pascal[depth - 1][idx]) % 10007;
        }

        pascal[depth][depth] = 1; // 마지막 원소
    }

    // 결과 출력
    console.log((pascal[N - 1][K - 1] + pascal[N - 1][K]) % 10007);
}
```

**문제[079] 다리 놓기**

시간 제한 0.5초 | 난이도 골드 V | 백준 온라인 저지 1010번

강 주변에서 다리를 짓기에 적합한 곳을 사이트라고 한다. 강의 서쪽에는 N개, 동쪽에는 M개의 사이트가 있다($N \leq M$). 서쪽의 사이트와 동쪽의 사이트를 다리로 연결하려고 한다. 이때 한 사이트에는 최대 1개의 다리만 연결될 수 있다.

다리를 최대한 많이 지으려고 하므로 서쪽의 사이트 개수 N만큼 다리를 지으려고 한다. 다리끼리는 서로 겹쳐질 수 없다고 할 때 다리를 지을 수 있는 경우의 수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 테스트 케이스의 개수 T, 그다음 줄부터 각 테스트 케이스에서 사이트의 개수가 정수 N, M($0 < N \leq M < 30$)으로 주어진다.

↑ 출력

각 테스트 케이스에서 다리를 지을 수 있는 경우의 수를 출력한다.

예제 입력 1

```
3 // 질의 수
2 2 // N, M
1 5
13 29
```

예제 출력 1

```
1
5
67863915
```



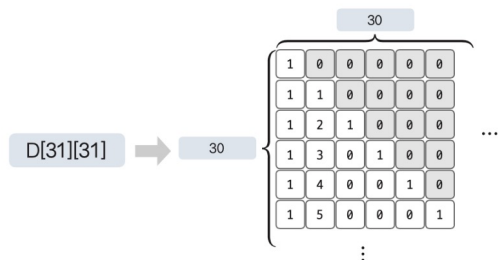
1 D[31][31] 로 DP 배열을 선언하고, 값을 다음 조건에 따라 초기화한다.

DP 배열 초기화

```

D[i][1] = i // i개 중 1개를 뽑는 경우의 수는 i개
D[i][0] = 1 // i개 중 1개도 선택하지 않는 경우의 수는 1개
D[i][i] = 1 // i개 중 i개를 선택하는 경우의 수는 1개

```

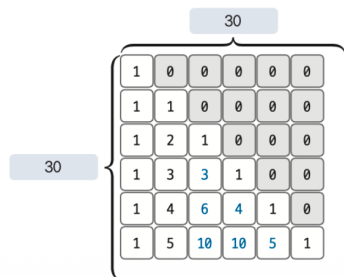


2 점화식을 이용해 DP 배열을 채운다.

N과 M의 최댓값이 30보다 작으므로 미리 D 배열의 값을 30까지 구한다.

조합의 점화식

$$D[i][j] = D[i-1][j] + D[i-1][j-1]$$



3 테스트 케이스를 실행해 D[M][N]의 값을 출력한다.

Python

```

import sys

input = sys.stdin.readline

# 1. T 입력
T = int(input())

# 2. N, M 입력
for _ in range(T):
    N, M = map(int, input().split())

    # 3. dp 테이블 정의
    D = [[0 for _ in range(M + 1)] for _ in range(N + 1)]

    # 4. dp 테이블 채우기
    for j in range(1, M+1):
        D[1][j] = j
        D[0][j] = 1

    for i in range(2, N+1):
        for j in range(i, M+1):
            D[i][j] = D[i-1][j-1] + D[i][j-1]

    # 5. 원하는 형식으로 출력
    print(D[N][M])

```

Java, JS

다리 놓기



Java

```

public class P006 { & kim-do-kyun
    public static void main(String[] args) throws IOException { & kim-do-kyun
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();

        // 1. T 입력
        int T = Integer.parseInt(br.readLine());

        for (int t = 0; t < T; t++) {
            // 2. N, M 입력
            StringTokenizer st = new StringTokenizer(br.readLine());
            int N = Integer.parseInt(st.nextToken());
            int M = Integer.parseInt(st.nextToken());

            // 3. dp 테이블 정의 (기본값 0으로 자동 초기화됨)
            int[][] D = new int[N + 1][M + 1];

            // 4. dp 테이블 채우기
            // 초기값 설정 (서쪽 사이트가 1개일 때, 0개일 때)
            for (int j = 0; j <= M; j++) {
                D[0][j] = 1; // 파이션 코드 로직 유지
                if (j > 0) D[1][j] = j;
            }

            // 점화식 적용
            for (int i = 2; i <= N; i++) {
                for (int j = i; j <= M; j++) {
                    D[i][j] = D[i - 1][j - 1] + D[i][j - 1];
                }
            }

            // 5. 결과 저장
            sb.append(D[N][M]).append('\n');
        }

        // 전체 출력
        System.out.print(sb);
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split(/\s+/);

// 입력 커서 관리
let cursor = 0;

// 1. T 입력
const T = parseInt(input[cursor++]);

for (let t = 0; t < T; t++) {
    // 2. N, M 입력
    const N = parseInt(input[cursor++]);
    const M = parseInt(input[cursor++]);

    // 3. dp 테이블 정의
    // (N+1) x (M+1) 크기의 2차원 배열 생성 및 0으로 초기화
    const D = Array.from({ length: N + 1 }, () => new Array(M + 1).fill(0));

    // 4. dp 테이블 채우기
    for (let j = 1; j <= M; j++) {
        D[1][j] = j;
        D[0][j] = 1;
    }

    for (let i = 2; i <= N; i++) {
        // j는 i부터 M까지
        for (let j = i; j <= M; j++) {
            D[i][j] = D[i - 1][j - 1] + D[i][j - 1];
        }
    }

    // 5. 원하는 형식으로 출력
    console.log(D[N][M]);
}

```

Python

순열의 순서 구하기



빈출

문제[081] 순열의 순서 구하기

시간 제한 2초 | 난이도 골드 V | 백준 온라인 저지 1722번

1부터 N까지의 수를 임의로 배열한 순열의 경우의 수는 N!이다. 임의의 순열은 영문 사전의 정렬 방식과 비슷하게 정렬된다고 하자. 예를 들어 $N = 3$ 이면 {1, 2, 3}, {1, 3, 2}, {2, 1, 3}, {2, 3, 1}, {3, 1, 2}, {3, 2, 1}의 순서로 정렬된다. N이 주어지면 다음 두 소문제 중 1개를 푸는 프로그램을 작성해 보자.

- 소문제 ① K가 주어지면 K번째 순열을 구한다.
- 소문제 ② 임의의 순열이 주어지면 이 순열이 몇 번째 순열인지 구한다.

↓ 입력

1번째 줄에 순열의 개수 N ($1 \leq N \leq 20$), 2번째 줄의 1번째 수는 소문제 번호다. 1일 때 K를 입력받고, 2일 때 임의의 순열을 나타내는 N개의 수를 입력받는다. 여기서 N개의 수에는 1에서 N까지의 정수가 한 번씩만 나타난다.

↑ 출력

주어진 소문제와 관련된 정답을 출력한다.

예제 입력 1

```
4 // 자릿수 N
1 3 // 소문제 번호, K
```

예제 출력 1

```
1 3 2 4
```

예제 입력 2

```
4 // 자릿수 N
2 1 3 2 4 // 소문제 번호, 순열
```

예제 출력 2

```
3
```



Python

순열의 순서 구하기 - 정답

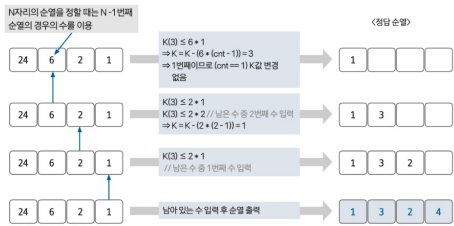
1 자릿수에 따른 순열의 경우의 수를 1부터 N자리까지 미리 계산한다.



2 소문제 ①을 풀어보자. 예제 1을 이용해 K번째 순열을 출력한다.

K번째 순열 출력하기

- ① 주어진 값(K)과 현재 자리(N) - 1에서 만들 수 있는 경우의 수를 비교한다.
- ② ①에서 K가 더 작아질 때까지 경우의 수를 백수(cnt)로 증가시킨다(순열의 순서를 1씩 늘림).
- ③ K가 더 작아지면 순열에 값을 저장하고 K를 K - (경우의 수 * (cnt - 1))로 업데이트한다.
- ④ 순열이 완성될 때까지 ①~③을 반복하고 완료된 순열을 출력한다.



3 소문제 ②을 풀어보자. 예제 2를 이용해 입력된 순열의 순서 K를 구한다.

입력된 순열의 순서 구하기

- ① N자리 순열의 숫자를 받아 몇 번째 순서의 숫자인지 확인한다(현재 숫자 - 자기보다 앞 숫자 중 이미 사용한 숫자).
- ② 해당 순서 * (현재 자리 - 1에서 만들 수 있는 순열의 개수)를 K에 더한다.
- ③ 모든 자릿수에 관해 ①~②를 반복한 후 K값을 출력한다.



Python

```
import sys
input = sys.stdin.readline

N = int(input())
c = list(map(int, input().split()))

def fact(i):
    if i <= 1:
        return 1
    else:
        return i * fact(i-1)

if c[0] == 1:
    k = c[1] - 1
    num = list(range(1, N+1))
    tmp = []

    while N > 0:
        f = fact(N-1)
        idx = k // f
        tmp.append(num.pop(idx))
        k %= f
        N -= 1

    print(" ".join(map(str, tmp)))

elif c[0] == 2:
    data = c[1:]
    num = list(range(1, N+1))
    rs = 0

    for i in range(N):
        f = fact(N-i-1)
        idx = num.index(data[i])
        rs += idx * f
        num.pop(idx)

    print(rs + 1)
```


Java, JS

순열의 순서 구하기



Java

```
public class P007 {  Ⓜ kim-do-kyun
    public static void main(String[] args) throws IOException {  Ⓜ kim-do-kyun
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int N = Integer.parseInt(br.readLine());
        StringTokenizer st = new StringTokenizer(br.readLine());

        int type = Integer.parseInt(st.nextToken());

        // 1부터 N까지 숫자를 관리할 리스트 (Python의 num)
        ArrayList<Integer> num = new ArrayList<>();
        for (int i = 1; i <= N; i++) {
            num.add(i);
        }

        // 팩토리얼 값을 미리 계산하거나 함수로 사용 (20!은 long 범위)
        // 여기서는 간단하게 함수 호출 방식 사용

        if (type == 1) {
            long K = Long.parseLong(st.nextToken());
            K -= 1; // 0-index 보정

            StringBuilder sb = new StringBuilder();

            // N부터 1까지 감소하며 처리
            for (int i = N; i > 0; i--) {
                long f = factorial((int) i - 1);

                // 몇 번째 구간인지 확인
                int idx = (int) (K / f);

                // 해당 숫자 가져오기 및 리스트에서 제거
                sb.append(num.get(idx)).append(" ");
                num.remove(idx);

                K %= f;
            }

            System.out.println(sb.toString().trim());
        }
    }
}
```

```
    } else if (type == 2) {
        int[] inputData = new int[N];
        for (int i = 0; i < N; i++) {
            inputData[i] = Integer.parseInt(st.nextToken());
        }

        long result = 0;

        for (int i = 0; i < N; i++) {
            long f = factorial((int) N - 1 - i);

            // 현재 숫자가 남은 숫자들 중 몇 번째인지(index) 찾기
            int idx = num.indexOf(inputData[i]);

            result += idx * f;
            num.remove(idx); // 사용한 숫자 제거
        }

        // 1-index 복구 후 출력
        System.out.println(result + 1);
    }
}

// 팩토리얼 함수 (long 반환 필수)
public static long factorial(int n) {  2 usages  Ⓜ kim-do-kyun
    long val = 1;
    for (int i = 2; i <= n; i++) {
        val *= i;
    }

    return val;
}
}
```



JS

• 순열의 순서 구하기 - 정답

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split(/\s+/);

let cursor = 0;
const N = parseInt(input[cursor++]);
const type = parseInt(input[cursor++]);

// 팩토리얼 함수 (BigInt 사용)
function getFactorial(n) {
    let val = 1n;
    for (let i = 2n; i <= BigInt(n); i++) {
        val *= i;
    }
    return val;
}

// 1~N 숫자 리스트 생성
let num = [];
for (let i = 1; i <= N; i++) num.push(i);

if (type === 1) {
    // 입력받은 K를 BigInt로 변환
    let K = BigInt(input[cursor++]);
    K -= 1n; // 0-index

    let result = [];

    for (let i = N; i > 0; i--) {
        // (i-1)! 계산
        let f = getFactorial(i - 1);

        // 몫 구하기 (BigInt 나눗셈은 소수점 버림)
        let idx = K / f;

        // 배열 인덱스로 쓰기 위해 Number로 변환
        let idxNum = Number(idx);

        result.push(num[idxNum]);
        num.splice(idxNum, 1); // 해당 인덱스 요소 제거

        K %= f;
    }

    console.log(result.join(" "));
}
```

```
} else {
    // 문제 유형 2
    let data = [];
    for (let i = 0; i < N; i++) {
        data.push(parseInt(input[cursor++]));
    }

    let rs = 0n; // 결과값도 BigInt

    for (let i = 0; i < N; i++) {
        let f = getFactorial(N - 1 - i);

        // 현재 숫자의 인덱스 찾기
        let idx = num.indexOf(data[i]);

        // 계산
        rs += BigInt(idx) * f;

        // 사용한 숫자 제거
        num.splice(idx, 1);
    }

    // 결과에 1 더해서 출력 (String 변환 권장)
    console.log((rs + 1n).toString());
}
```

감사합니다!